

Parallel Stencil Computation

Arahí Fernández Monagas
University of Trieste
High-Performance Computing
2024-2025

October 10, 2025

The Assignment: Simulating the 2D Heat Equation

The project goal was to parallelize a stencil computation code solving the 2D heat equation:

$$\frac{\partial u(t, \vec{x})}{\partial t} = \alpha \nabla^2 u(t, \vec{x})$$

The Assignment: Simulating the 2D Heat Equation

The project goal was to parallelize a stencil computation code solving the 2D heat equation:

$$\frac{\partial u(t, \vec{x})}{\partial t} = \alpha \nabla^2 u(t, \vec{x})$$

Core Objectives:

- A** Build a **parallel implementation** of the 5-point stencil based on the provided templates.

The Assignment: Simulating the 2D Heat Equation

The project goal was to parallelize a stencil computation code solving the 2D heat equation:

$$\frac{\partial u(t, \vec{x})}{\partial t} = \alpha \nabla^2 u(t, \vec{x})$$

Core Objectives:

- A** Build a **parallel implementation** of the 5-point stencil based on the provided templates.
- B** The parallelization must be a hybrid model using both **MPI** and **OpenMP**. OpenMP should be used for the local grid's update loop.

The Assignment: Simulating the 2D Heat Equation

The project goal was to parallelize a stencil computation code solving the 2D heat equation:

$$\frac{\partial u(t, \vec{x})}{\partial t} = \alpha \nabla^2 u(t, \vec{x})$$

Core Objectives:

- A** Build a **parallel implementation** of the 5-point stencil based on the provided templates.
- B** The parallelization must be a hybrid model using both **MPI** and **OpenMP**. OpenMP should be used for the local grid's update loop.
- C** **Instrument the code** to measure the time spent on **computation** versus **communication**.

The Assignment: Simulating the 2D Heat Equation

The project goal was to parallelize a stencil computation code solving the 2D heat equation:

$$\frac{\partial u(t, \vec{x})}{\partial t} = \alpha \nabla^2 u(t, \vec{x})$$

Core Objectives:

- A** Build a **parallel implementation** of the 5-point stencil based on the provided templates.
- B** The parallelization must be a hybrid model using both **MPI** and **OpenMP**. OpenMP should be used for the local grid's update loop.
- C** **Instrument the code** to measure the time spent on **computation** versus **communication**.
- D** Perform a **scalability study** of the final implementation.

Parallelization Strategy: Domain Decomposition

From a Single Grid to a Grid of Tasks

Key Implementation Changes

- **Domain Decomposition:** The global domain is split among MPI tasks.
- **Neighbor Finding:** Each process identifies its N, S, E, W neighbors.
- **Memory Layout:** The grid is expanded to include extra layers for “halos” (ghost cells).

Parallelization Strategy: Domain Decomposition

From a Single Grid to a Grid of Tasks

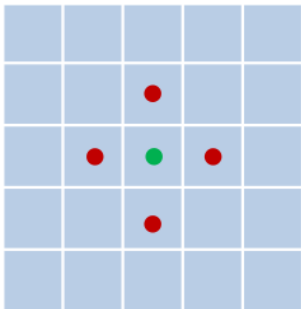


Figure: The new value of the green point depends on the four red points.

Parallelization Strategy: Domain Decomposition

From a Single Grid to a Grid of Tasks

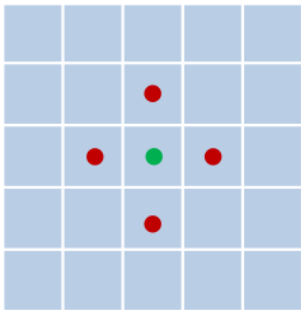


Figure: The new value of the green point depends on the four red points.

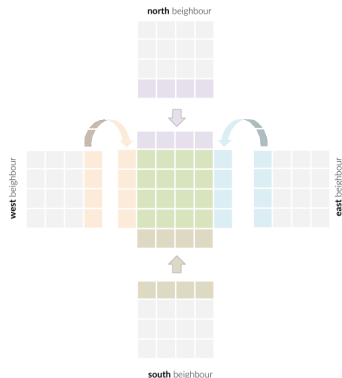


Figure: Domain decomposition between two MPI tasks, showing the need for communication at the boundary.

The New Main Loop

Overlapping Communication and Computation

The simple serial loop was replaced with a multi-stage process to manage communication and computation efficiently.

```
1 for (iter = 0; iter < N; ++iter)
2 {
3     // 1. Start non-blocking communication
4     exchange_halos(...);
5
6     // 2. Compute what you can (interior)
7     update_interior(...);
8
9     // 3. Wait for communication to finish
10    MPI_Waitall(...);
11
12    // 4. Compute the rest (borders)
13    update_borders(...);
14
15    // 5. Swap planes
16    current = !current;
17 }
```

This new structure allows the CPU to perform useful calculations while data arrives from other processes.

Key Implemented Functions

Separating Communication and Computation

Halo Exchange (Communication)

```
void exchange_halos(...);
```

Purpose: Manages all MPI communications by posting non-blocking sends and receives to overlap data transfer with computation.

Key Implemented Functions

Separating Communication and Computation

Halo Exchange (Communication)

```
void exchange_halos(...);
```

Purpose: Manages all MPI communications by posting non-blocking sends and receives to overlap data transfer with computation.

Stencil Update (Computation)

```
void update_interior(...);
```

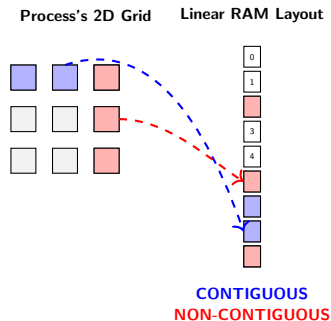
```
void update_borders(...);
```

Purpose: Splits the update into two stages: one for the inner (safe) points and another for the boundary points that depend on communication.

Dissecting the `exchange_halos` Function

The Choreography of Non-Blocking Communication

- The goal is to post all halo exchanges using non-blocking MPI calls (`Isend/Irecv`).
- **North/South data** is **contiguous** in memory. This is simple and fast for MPI to handle.
- **East/West data** is **non-contiguous**. Its elements are scattered across memory.
- **The Solution:** A special `MPI_Datatype` is used to describe the memory gaps, allowing for efficient access without slow, manual data copying.

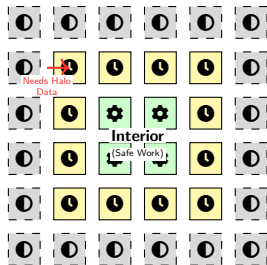


Splitting the Workload: Safe vs. Dependent Computation

The Logic Behind `update_interior` and `update_borders`

To overlap communication with computation, the update is split into two functions:

- `update_interior`:
 - Computes the grid's inner points.
 - This is **safe work**, as it does not depend on neighbor data.
 - Runs **during** communication to hide latency.

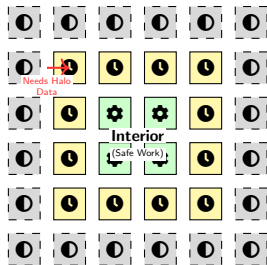


Splitting the Workload: Safe vs. Dependent Computation

The Logic Behind `update_interior` and `update_borders`

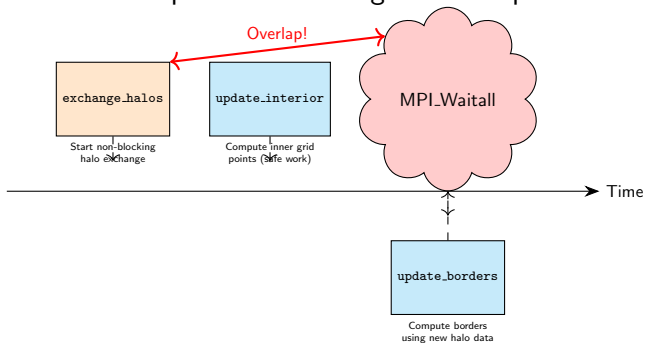
To overlap communication with computation, the update is split into two functions:

- `update_interior`:
 - Computes the grid's inner points.
 - This is **safe work**, as it does not depend on neighbor data.
 - Runs **during** communication to hide latency.
- `update_borders`:
 - Computes the grid's boundary points.
 - This is **dependent work**, as it requires the freshly received halo data.
 - Runs only **after** communication is complete (`MPI_Waitall`).



Graphical Explanation of the Loop

This is how the functions work together to overlap communication and computation in a single time step:



1. Start non-blocking halo exchange.
2. While data is in transit, compute the interior points.
3. Wait for communication to complete.
4. Compute the borders with the newly received data.

Hybrid Parallelization with OpenMP

Why Add OpenMP?

The strategy combines two models to maximize the use of the hardware:



Node 1: OpenMP (Threads)

Node 2: OpenMP (Threads)

Hybrid Parallelization with OpenMP

Why Add OpenMP?

The strategy combines two models to maximize the use of the hardware:



Node 1: OpenMP (Threads)

Node 2: OpenMP (Threads)

- **MPI (Inter-Node):** Handles communication **between** different compute nodes across the network.
- **OpenMP (Intra-Node):** Uses multiple threads **within** a single node, sharing the same memory.

```
1 #pragma omp parallel for
2 for (uint j = 1; j <= sizeY; ++j) {
3     for (uint i = 1; i <= sizeX; ++i) {
4         // ... stencil calculation ...
5     }
6 }
```

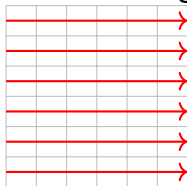
Optimization: OpenMP Tiling for Cache Efficiency

- **The Problem:** Processing row by row leads to frequent cache misses.

Optimization: OpenMP Tiling for Cache Efficiency

- **The Problem:** Processing row by row leads to frequent cache misses.

Without Tiling

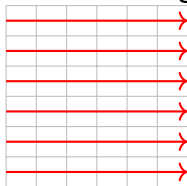


Poor data locality

Optimization: OpenMP Tiling for Cache Efficiency

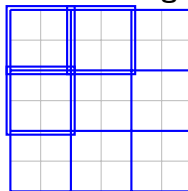
- **The Problem:** Processing row by row leads to frequent cache misses.

Without Tiling



Poor data locality

With Tiling

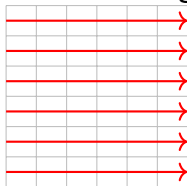


High cache reuse

Optimization: OpenMP Tiling for Cache Efficiency

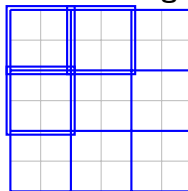
- **The Problem:** Processing row by row leads to frequent cache misses.

Without Tiling



Poor data locality

With Tiling



High cache reuse

- **The Solution:** The grid is divided into small square tiles that fit in the cache. OpenMP threads process one tile at a time.
- **The Benefit:** Data in a tile is reused extensively before being replaced, which significantly speeds up computation.

Code Instrumentation

Measuring Performance to Find Bottlenecks

To analyze performance, the main loop's total time was broken down into its two key components using `MPI_Wtime()`.

Total Execution Time

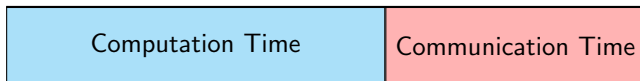


Figure: The total runtime of each iteration is the sum of the time spent doing useful work (computation) and the time spent waiting for data (communication).

This measurement allows us to identify which part is the bottleneck and how the **ratio** of computation vs. communication changes as we scale the problem.

Verification: 2D Heatmap Sequence

After all the strategies implemented, the visualizations confirmed the correct physical behaviour of the heat diffusion.

Time Steps 0, 10, 30, and 49:

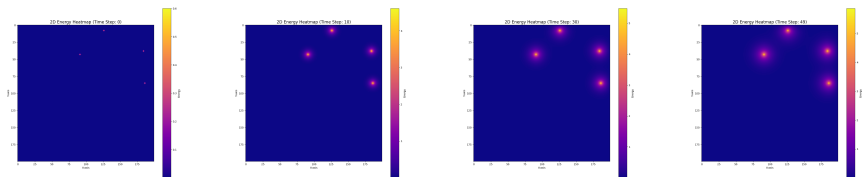


Figure: The energy clearly spreads outwards from the center over time

Performance Analysis: Methodology

Performance was evaluated on a cluster using SLURM batch scripts to automate three scaling studies, as required by the assignment.

- **Strong Scaling:** Measures speedup for a **fixed total problem size**.
- **Weak Scaling:** Measures scalability for a **problem size that grows with resources**.
- **OpenMP Scaling:** Measures speedup from threading on a **single node**.

Performance Analysis: Test Configuration

Scaling Parameters

Table: Parameters for the three scaling studies.

Parameter	Strong Scaling	Weak Scaling
Problem Size	Fixed: 15000 × 15000	Grows with nodes
Work per Node	Decreases	Fixed: 10000 × 10000
Iterations	500	500
MPI Tasks/Node	8	8
OMP Threads/Task	14	14
Node Count	1, 2, 4, 8, 16	1, 2, 4, 8, 16

OpenMP Scaling (Single Node)

Configuration: A separate test on a 2000x2000 grid for 100 iterations, scaling from 1 to 112 threads.

Results: MPI Strong Scaling

Fixed problem size, increasing processors.

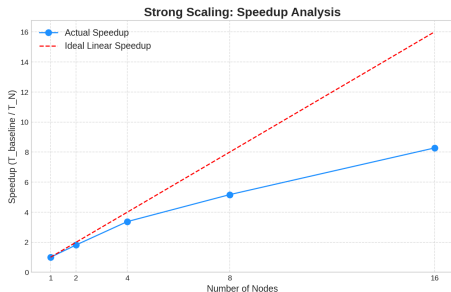


Figure: Speedup

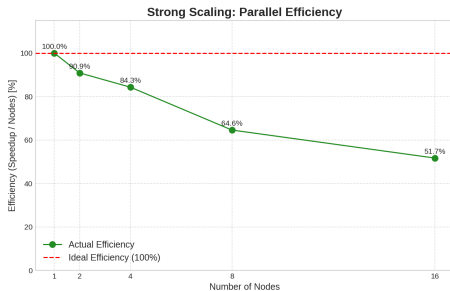


Figure: Efficiency

Analysis: MPI Strong Scaling

- The speedup is nearly linear, demonstrating effective parallelization.
- As predicted by Amdahl's Law, efficiency decreases as more processors are added.

$$\text{Speedup}(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

- This happens because the fixed communication overhead becomes more significant as the computational work per process shrinks.

Results: MPI Weak Scaling

Constant work per processor, increasing problem size.

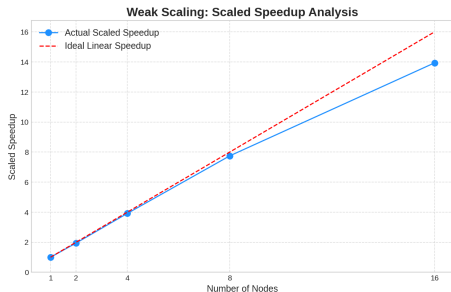


Figure: Speedup

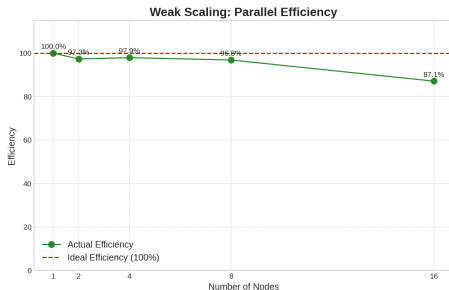


Figure: Efficiency

Analysis: MPI Weak Scaling

- This is an excellent result, fulfilling a key goal of scalable computing.
- The efficiency remains high and relatively flat (around 80-90%), indicating the code is highly scalable.
- It proves the implementation can effectively solve larger problems by adding more resources without being dominated by communication overhead.

Total time Strong vs Weak

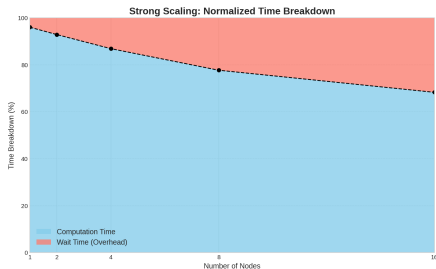


Figure: Performance (Absolute Time strong scalability)

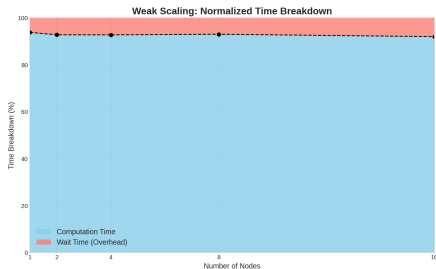


Figure: Performance (Absolute Time weak scalability)

Results: OpenMP Scaling on a Single Node

Fixed problem size, increasing threads.

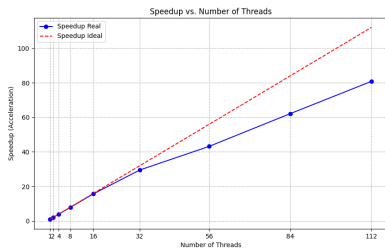


Figure: Speedup

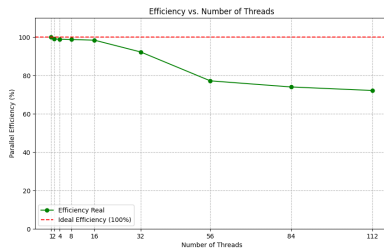


Figure: Efficiency

Analysis: OpenMP Scaling on a Single Node

- The code shows good speedup from threading, validating the shared-memory component of the hybrid model
- The drop in efficiency at higher thread counts is expected, likely due to thread management overhead and contention for shared resources like memory bandwidth.

Analysis: OpenMP Scaling on a Single Node

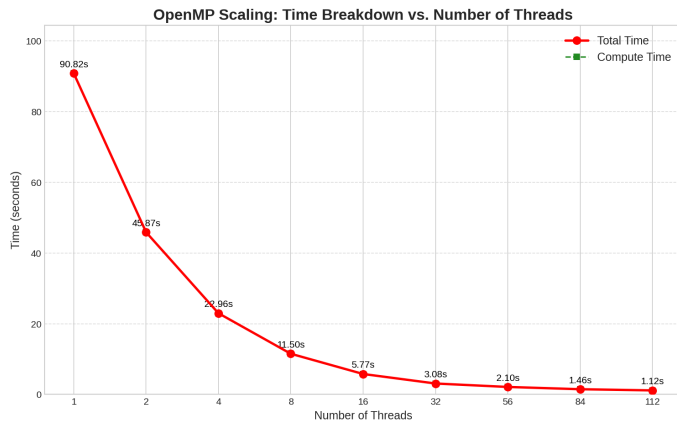


Figure: Total time

Summary of Key Challenges & Solutions

- **Low Computational Load:** `Wait_Time` dominated execution, causing poor scaling.
 - *Solution:* An “artificial workload” was added to increase `Compute_Time` and overlap communication.
- **Inefficient Cache Usage:** Suboptimal memory access limited the pure computation speed.
 - *Solution:* The “Cache Blocking” (Tiling) technique was implemented to improve data locality.

Conclusion

- A hybrid MPI/OpenMP stencil code was successfully parallelized, optimized, and verified, meeting all project requirements.
- Key optimizations like **overlapping communication** in *exchange_halos* and **tiling** in 'stencil_computation' were critical for performance.
- The performance analysis demonstrates a robust and efficient implementation with excellent weak scaling, a key indicator of a well-designed parallel application.

Future Work & Potential Improvements

- **GPU Acceleration:** Port the stencil computation kernel to CUDA or OpenACC to leverage the massive parallelism of GPUs.
- **Load Balancing:** For non-uniform grids, implement a dynamic load balancing scheme to distribute work more evenly.

Code Repository

The complete source code for this project is available on GitHub:

 <https://github.com/afm719/hpc25arahi>

Questions?

Thank You